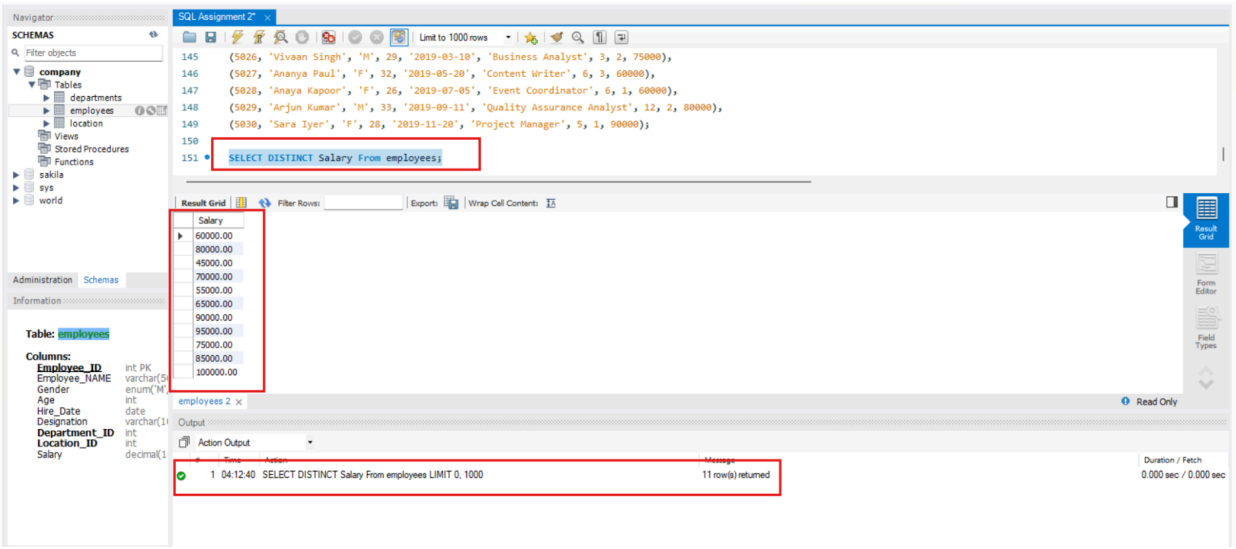
MySQL Assignment 2

Clause & Operators

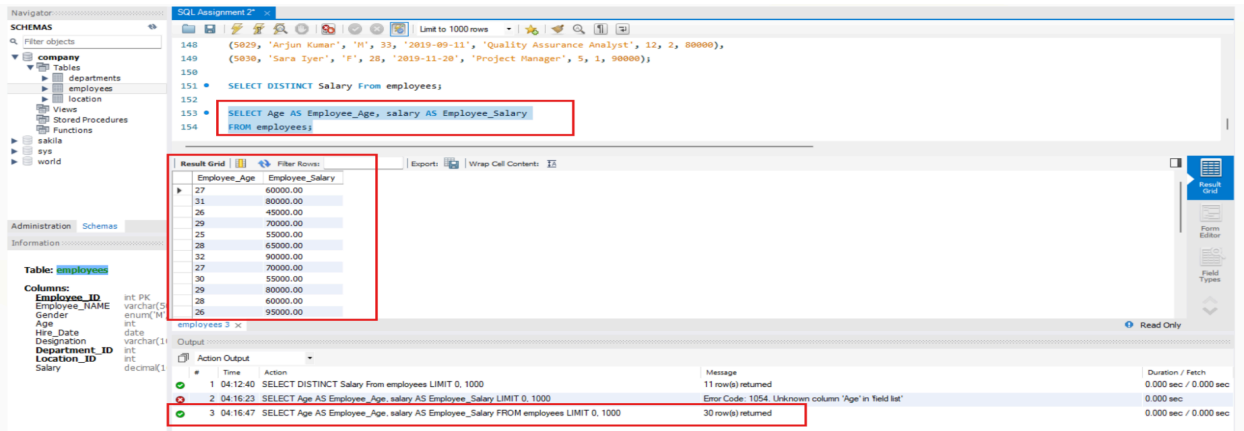
1. DISTINCT VALUES:

A query was created to retrieve distinct salaries from the Employees table.



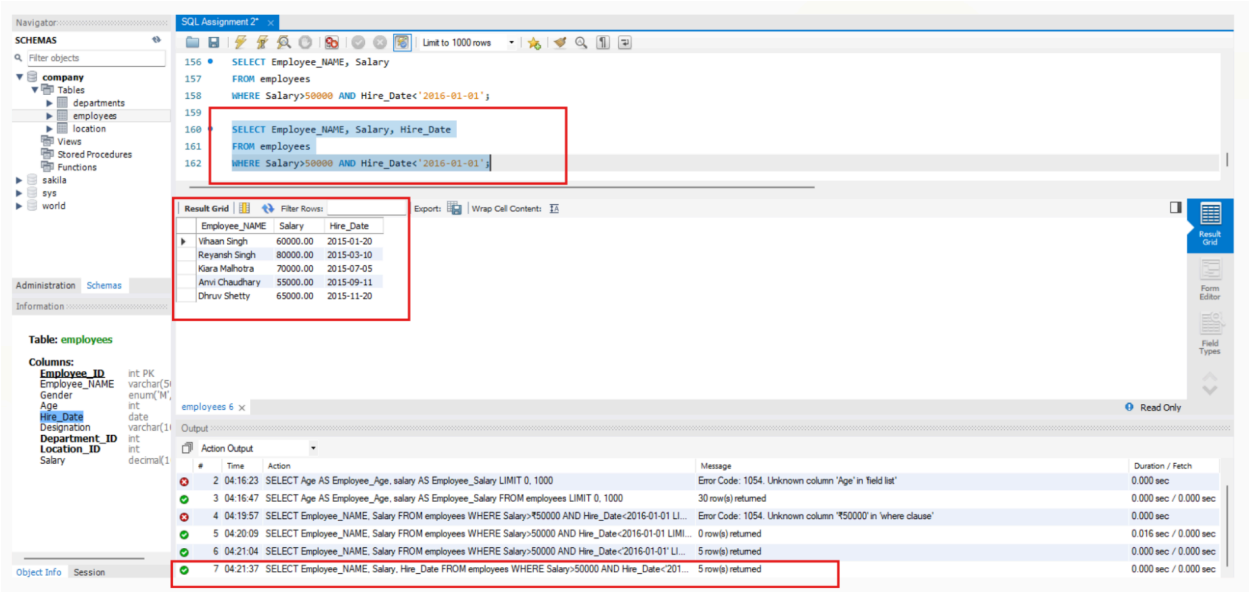
2. ALIAS (AS):

A query was created to assign aliases to the Age and Salary columns as `Employee_Age` and `Employee_Salary`, respectively.

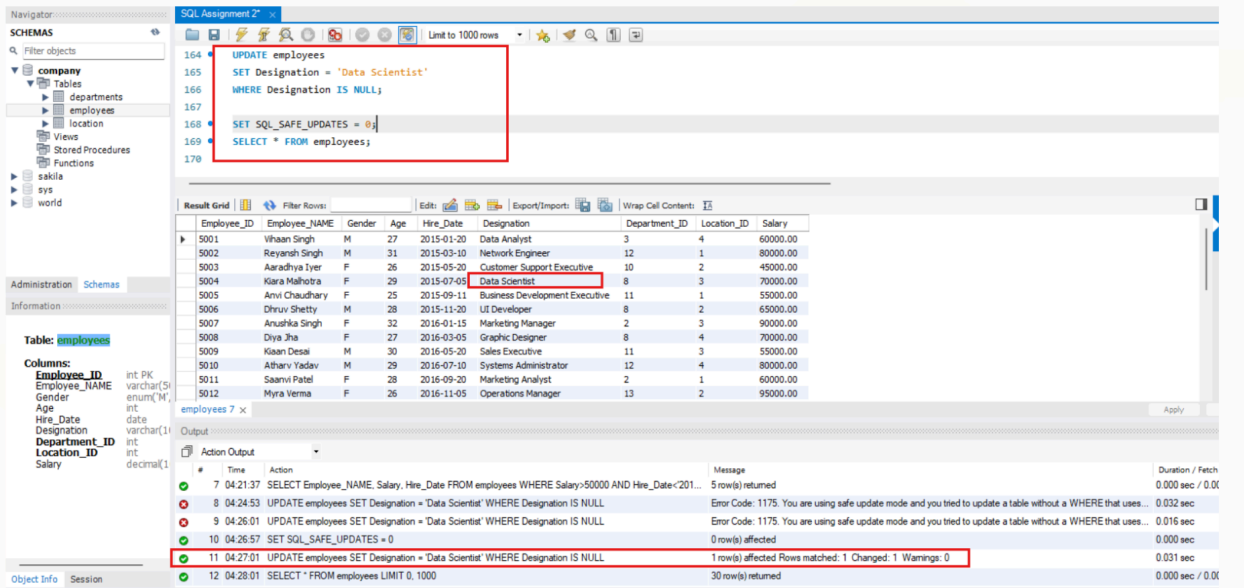


3. WHERE CLAUSE & OPERATORS:

● A query was created to retrieve employees with a salary greater than ₹50,000 who were hired before 2016-01-01.



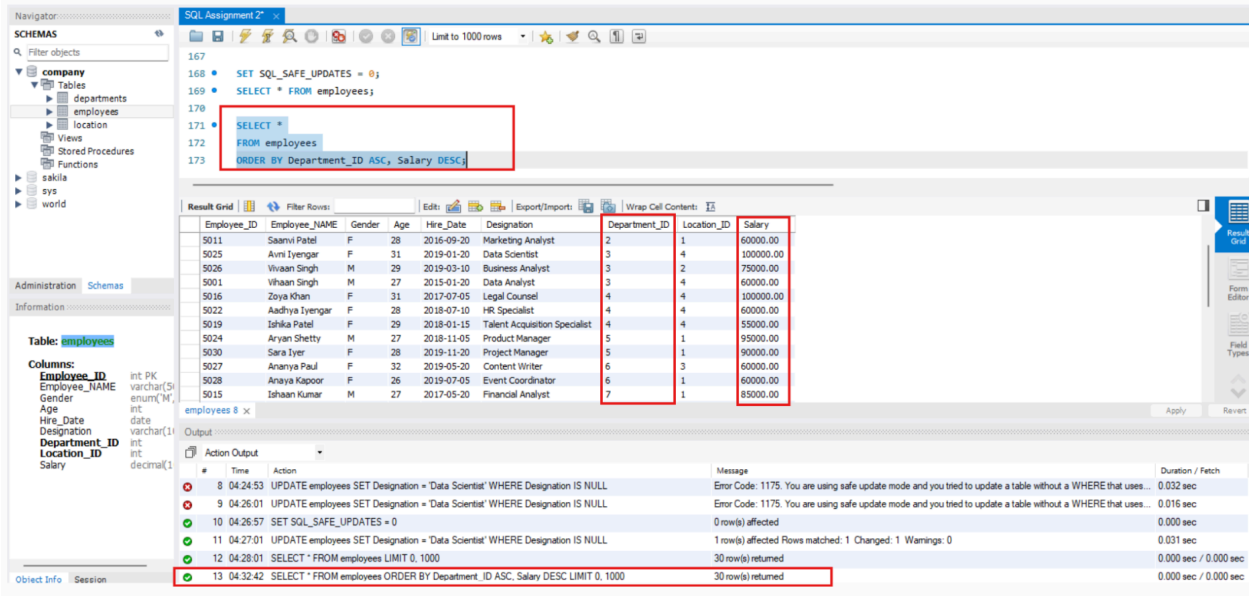
- The missing designation was identified and updated to **"Data Scientist"**.



Sorting and Grouping Data

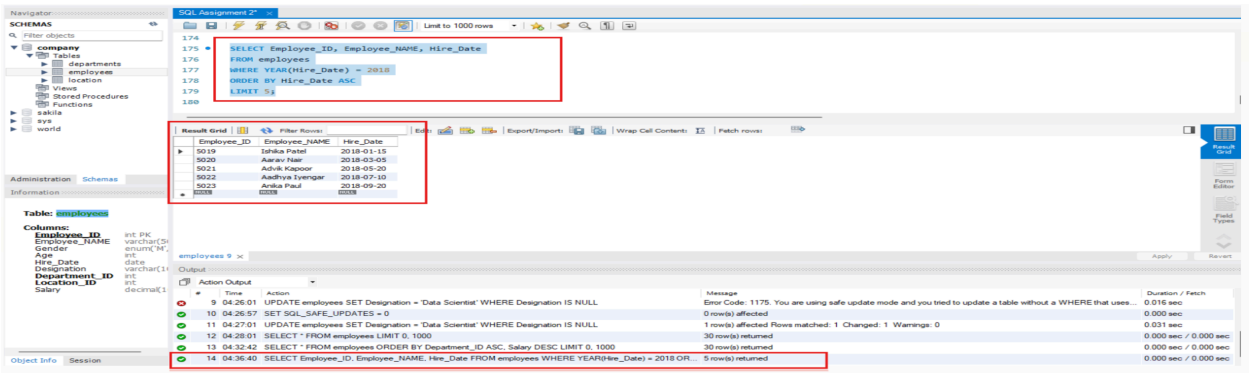
1. ORDER BY:

● A query was created to sort employees by Department ID in ascending order and Salary in descending order.



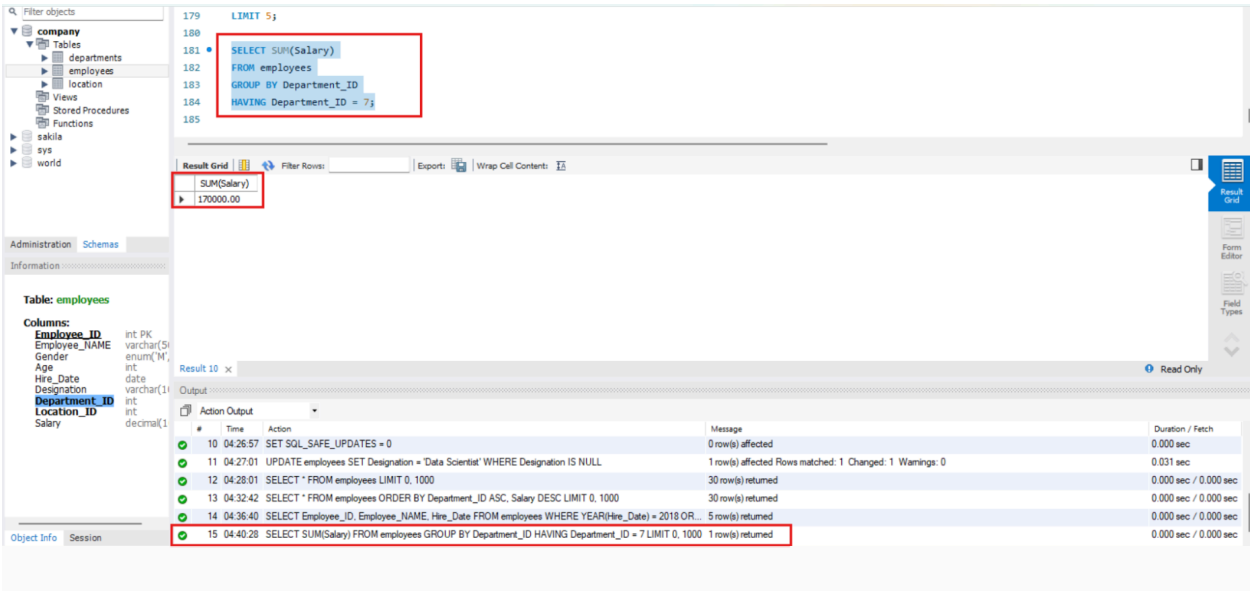
2. LIMIT:

● A query was created to display the first 5 employees who were hired in the year 2018.

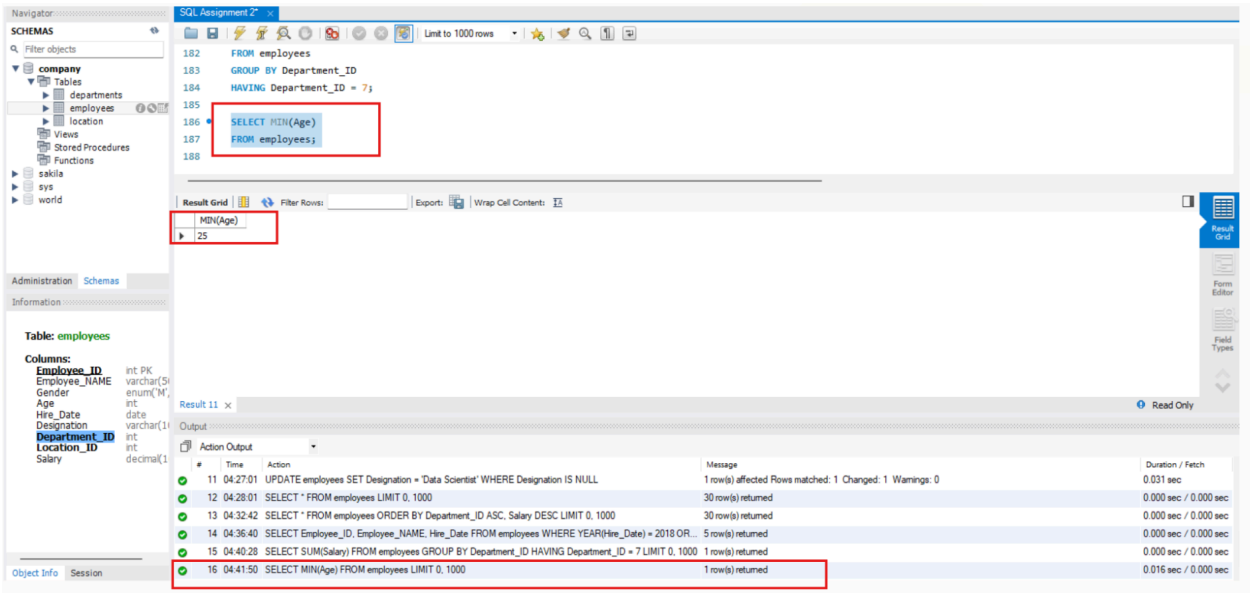


3. AGGREGATE FUNCTIONS:

● A query was created to calculate the total salary of employees in the Finance department.

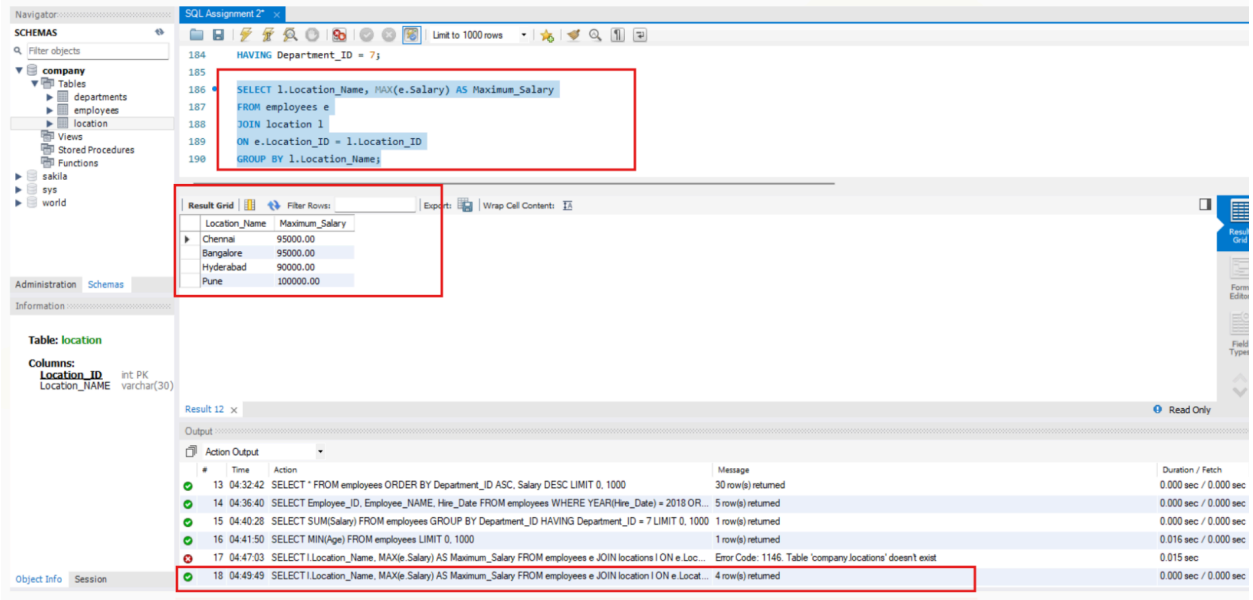


- A query was created to find the minimum age among all employees.

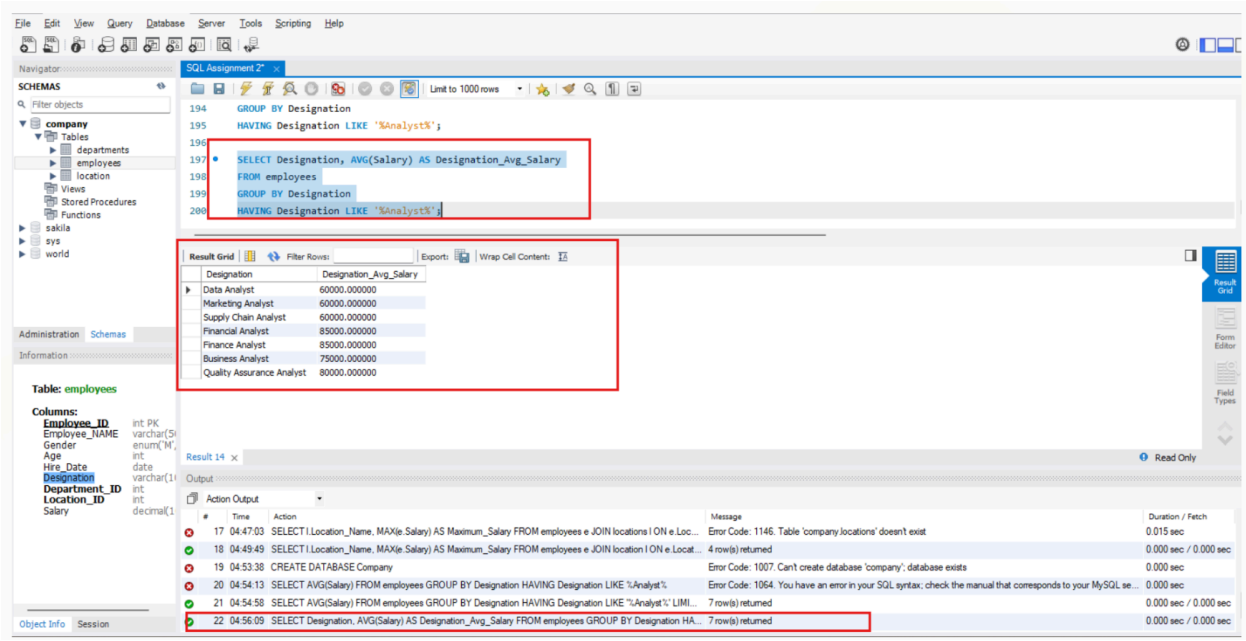


GROUP BY:

- A query was created to find the maximum salary for each location using `GROUP BY Location`.

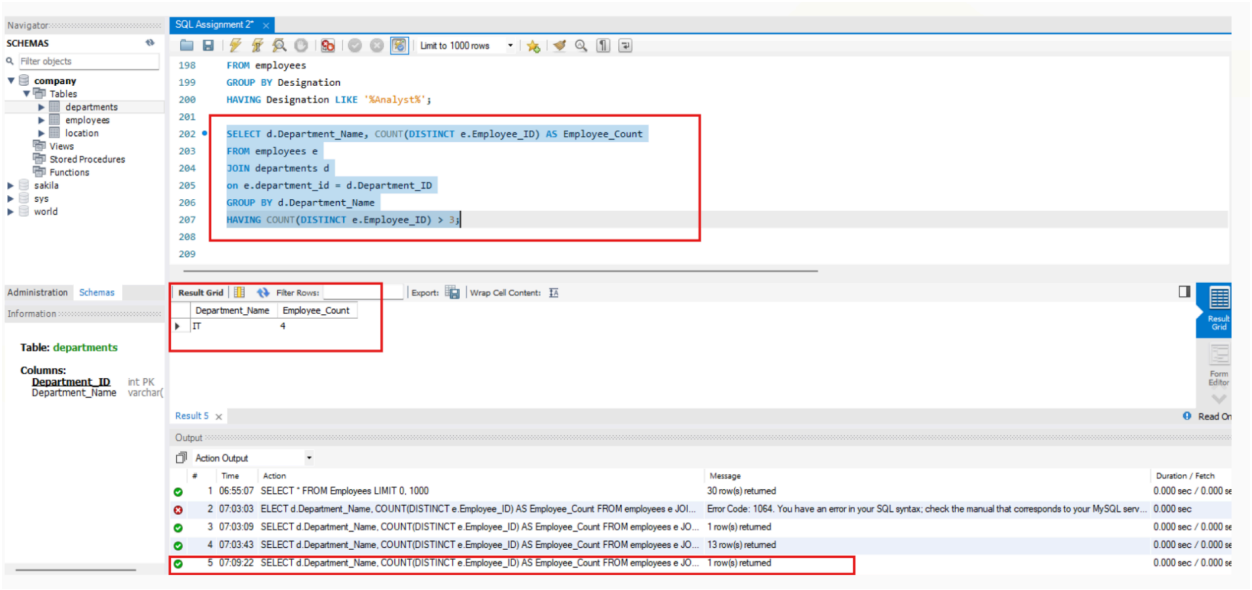


A query was created to calculate the average salary for each designation containing the word **'Analyst'**, using `GROUP BY Designation` and `HAVING Designation LIKE '%Analyst%'`.

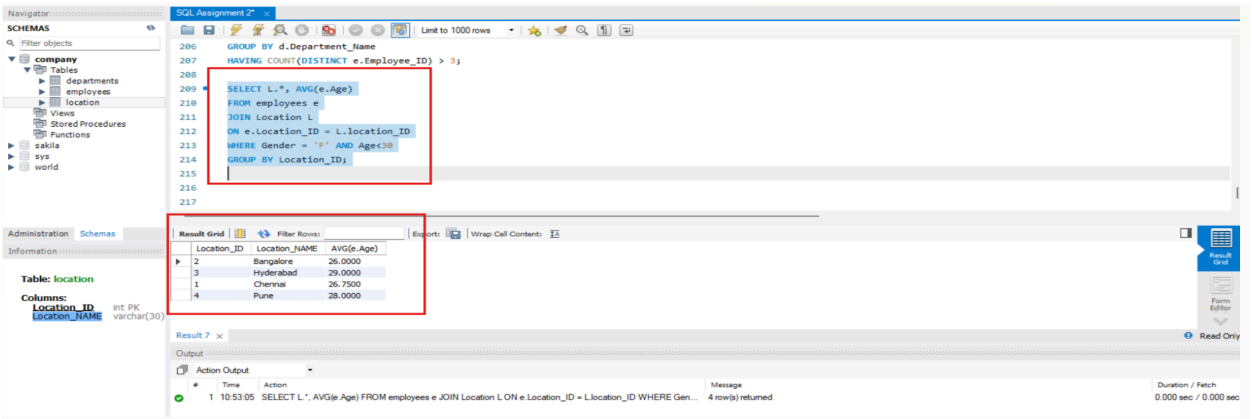


5. HAVING:

● A query was created to find departments with fewer than 3 employees using `GROUP BY Department` and `HAVING COUNT(Employee_ID) < 3`.



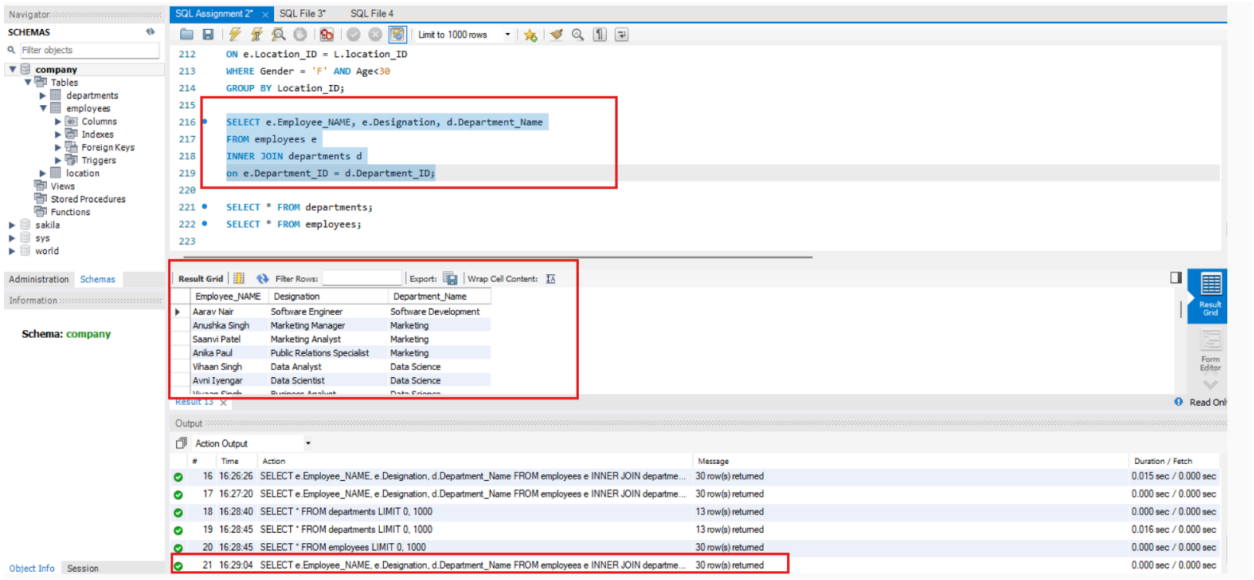
- A query was created to find locations where female employees have an average age below 30 using `WHERE Gender = 'F'`, `GROUP BY Location`, and `HAVING AVG(Age) < 30`.



Joins:

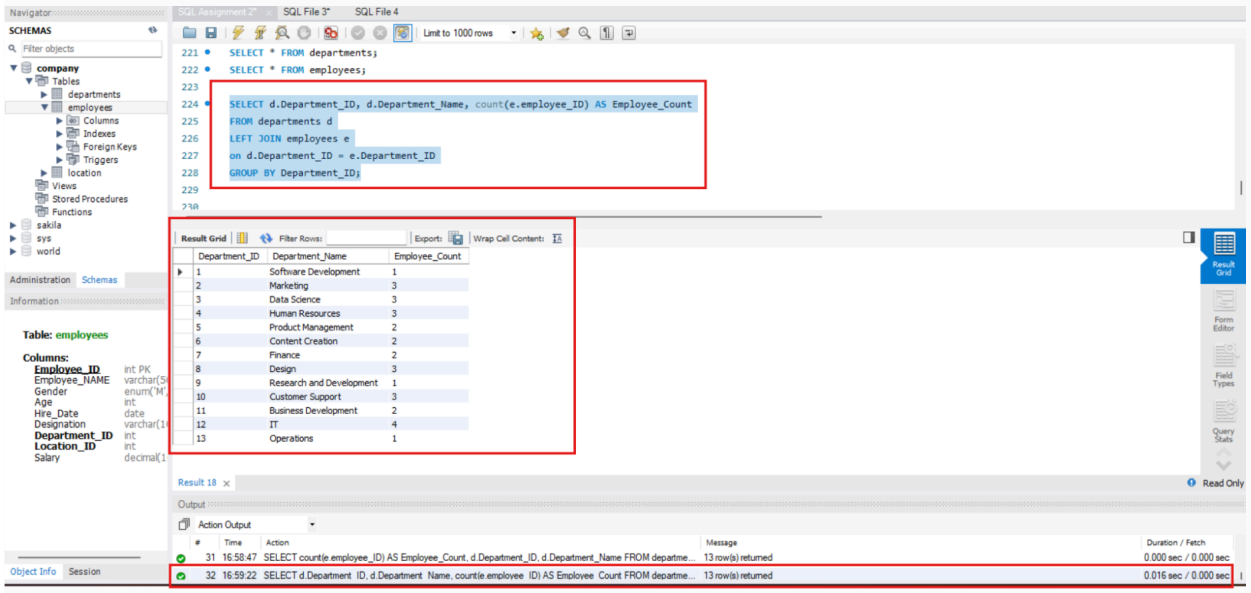
1. INNER JOIN:

● A query was created to retrieve employee names, designations, and department names for employees who are assigned to a department, using an `INNER JOIN` between the Employees and Departments tables.



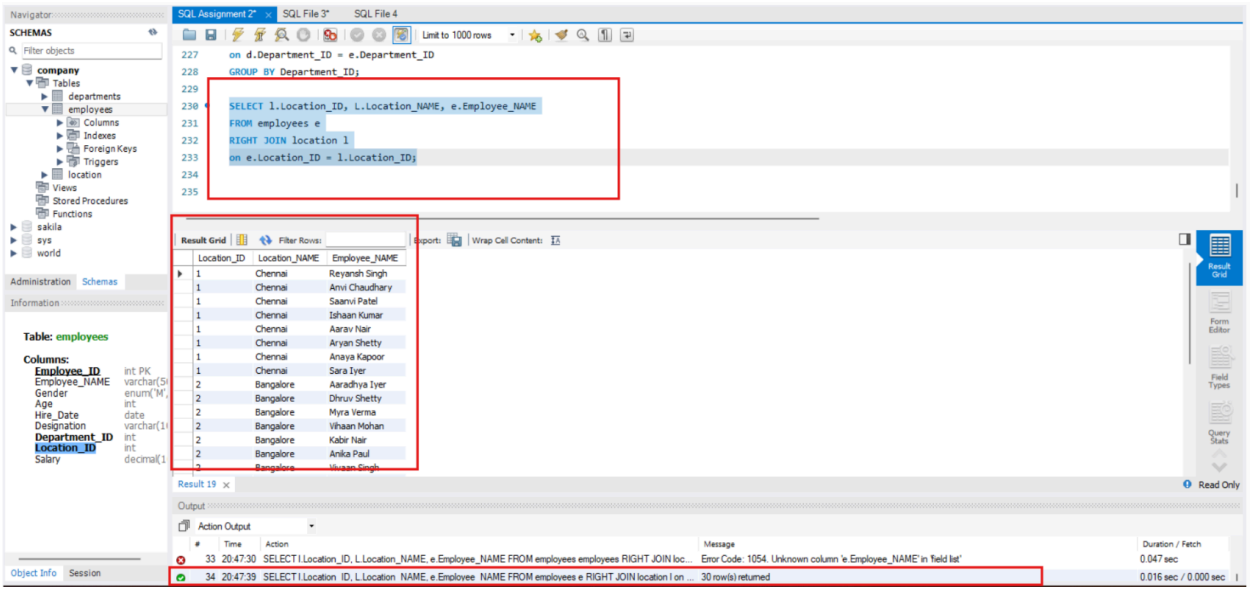
2. LEFT JOIN:

●A query was created to list all departments with the total number of employees in each department, including departments with no employees, using a `LEFT JOIN` and `COUNT()`.



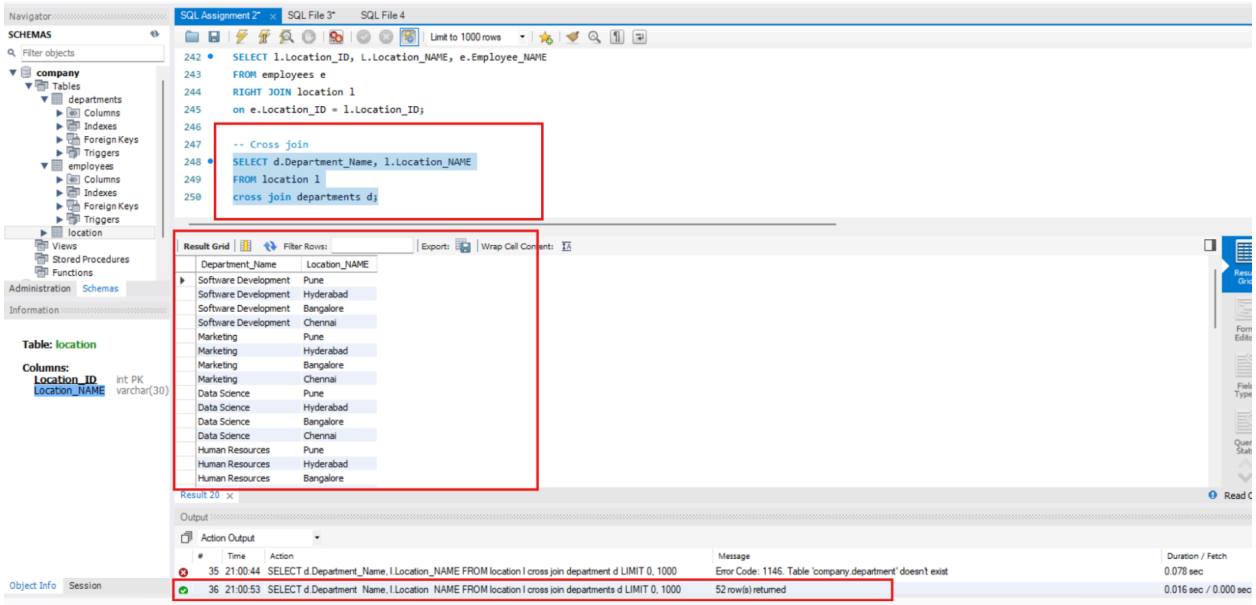
3. RIGHT JOIN:

● A query was created to display all locations along with the names of employees assigned to each location, including locations with no employees by using a `LEFT JOIN`.



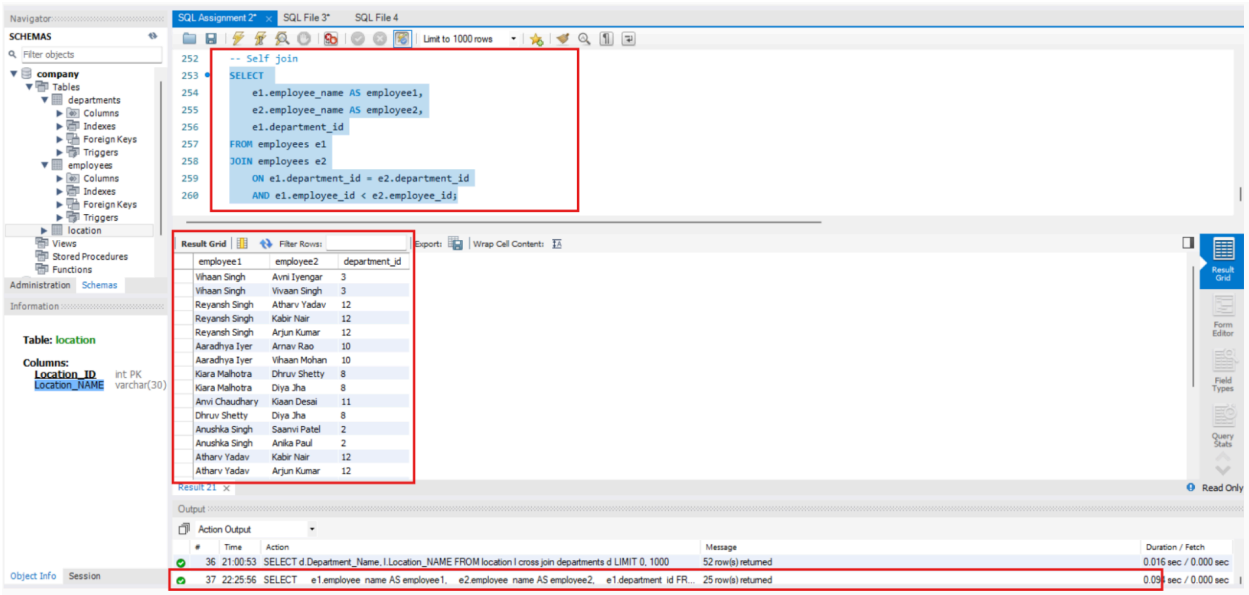
4. CROSS JOIN

● A query was created to display all possible combinations of departments and locations using a `CROSS JOIN`.



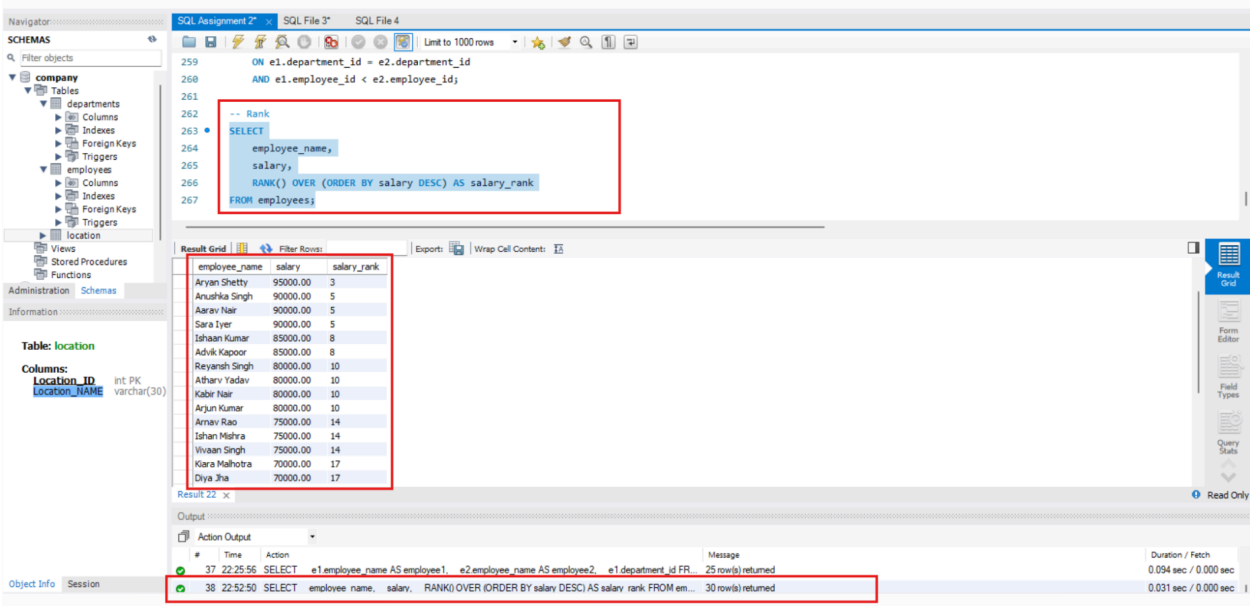
5. SELF JOIN:

● A query was created to display pairs of employees working in the same department, excluding self-pairs, using a **self-join** with `e1.employee_id < e2.employee_id`.

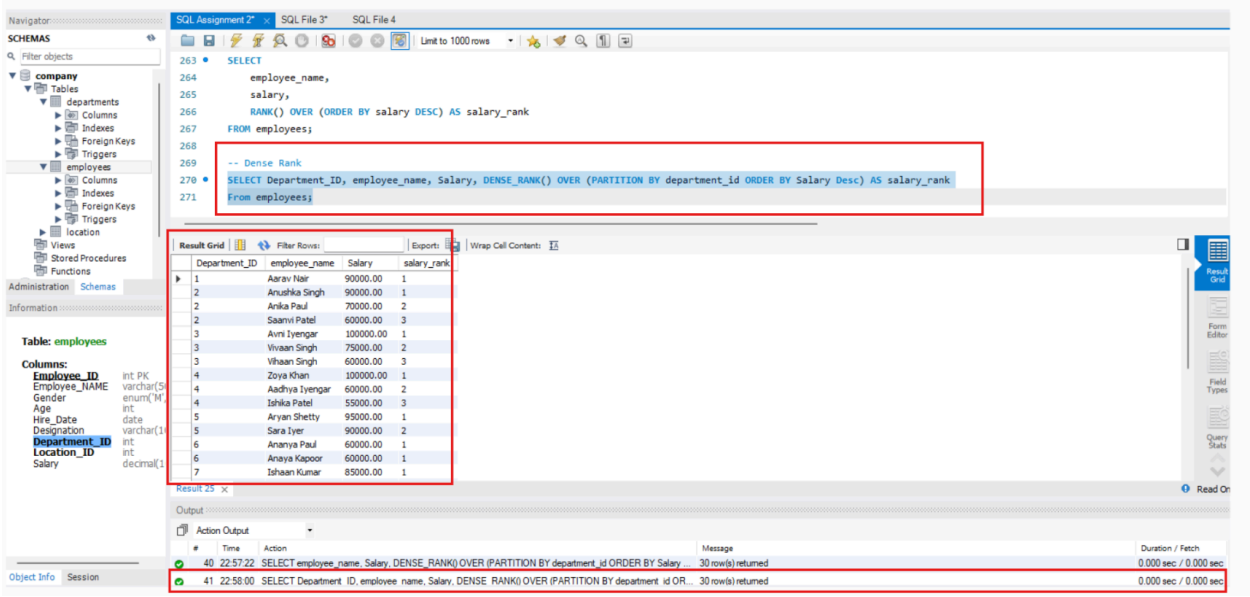


Windows function

●A query was created to rank employees by salary using the `RANK()` window function.



●A query was created to rank employees by salary within each department using the `DENSE_RANK()` window function.



● query was created to calculate the running total of salaries within each department using the `SUM()` window function with `PARTITION BY Department`.

